Connection Science

ISSN: 0954-0091 (Print) 1360-0494 (Online) Journal homepage: www.tandfonline.com/journals/ccos20

Analyzing execution path non-determinism of the Linux kernel in different scenarios

Yucong Chen, Xianzhi Tang, Shuaixin Xu, Fangfang Zhu, Qingguo Zhou & Tien-Hsiung Weng

To cite this article: Yucong Chen, Xianzhi Tang, Shuaixin Xu, Fangfang Zhu, Qingguo Zhou & Tien-Hsiung Weng (2023) Analyzing execution path non-determinism of the Linux kernel in different scenarios, Connection Science, 35:1, 2192442, DOI: 10.1080/09540091.2023.2192442

To link to this article: https://doi.org/10.1080/09540091.2023.2192442

Published online: 03 Apr 2023.

CONNECTION SCIENCE
2023, VOL. 35, NO. 1, 2192442
https://doi.org/10.1080/09540091.2023.2192442

Analyzing execution path non-determinism of the Linux kernel in different scenarios

Yucong Chen[a,b], Xianzhi Tang[a], Shuaixin Xu[a], Fangfang Zhu[b], Qingguo Zhou[a] and Tien-Hsiung Weng[c]

[a]School of Information Science and Engineering, Lanzhou University, Lanzhou, People's Republic of China; [b]Institute of Modern Physics, Chinese Academy of Sciences, Lanzhou, People's Republic of China; [c]Dept. of Computer Science and Information Engr. (CSIE), Providence University, Taichung, Taiwan

ABSTRACT

Safety-critical systems play a significant role in industrial domains, and their complexity is increasing with advanced technologies such as Artificial Intelligence (AI). To provide efficient services, safety-critical systems that integrate AI applications are always built based on Linux, where Linux offers massive amounts of features and an incredibly perfect software ecosystem for AI applications. Since Linux is a pre-existing complex software system, different research programmes aim to pave the way for developing Linux-based safety-critical systems. Still, only some focus on the system calls for file operations. However, the execution path of a system call is effectively non-deterministic in Linux kernel space, which challenges the test coverage-based verification recommended by the functional safety standards. This research analyzes the influence of system state on Linux kernel path variability from two perspectives: file system type and system load. Therefore, an online data collection system for system call execution paths was constructed based on Ftrace, network file system (NFS), and MD5 hash function, uniquely identifying the system call execution path. The collected data were processed and analysed in this study. Evaluations show that the number of function execution paths of the system calls relevant to file systems increased with the increase in system load but would eventually be stable. Additionally, the function execution paths of the system call varied in different file systems. Based on the evaluations, the results of this work can provide advice for analyzing Linux-based safety-critical systems. In addition, the method introduced in this research can also provide support for the verification of Linux-based safety-critical systems.

ARTICLE HISTORY
Received 14 December 2022
Accepted 14 March 2023

KEYWORDS
Linux kernel; system call; path variability; non-determinism

1. Introduction

As embedded systems' functionality, reliability, and performance continue to increase in many application domains, their complexity is also growing. Embedded systems in these domains are often safety-critical, some of which have high performance and advanced technology requirements (Tao et al., 2022). These systems usually integrate Free/Libre and Open

CONTACT Qingguo Zhou zhouqg@lzu.edu.cn; Tien-Hsiung Weng thweng@gm.pu.edu.tw

Source Software (FLOSS) components and Artificial Intelligence (AI) or Machine Learning (ML) algorithms (Diao et al., 2023; Manimuthu et al., 2022; Saddik et al., 2022; Yong et al., 2022; Zhang et al., 2022). In this context, such embedded systems require a general-purpose operating system (GPOS) to support these complex applications while also meeting the requirements of functional safety certification. For example, some advanced driver assistance systems (ADAS) currently use GNU/Linux and AI technology (Borrego-Carazo et al., 2020; Fedullo et al., 2022; Y. Li & Huang, 2022; Nidamanuri et al., 2021; Zhou et al., 2022). GNU/Linux has rich functional characteristics and a complete software ecological environment and supports real-time applications (Reghenzani et al., 2019, february), so it has a wide range of applications, from supercomputers to embedded systems. At the same time, more and more industries are developing various reliable and critical applications based on GNU/Linux (for example, electronic banking and telecommunications). As a result, different industry sectors sincerely evaluate whether Linux is suitable for their next-generation safety-critical systems (Hernandez et al., 2020). However, although many enterprises are ready to develop highly complex and reliable systems integrating Linux, there is still a need to pave the way for developing various reliable and safety-critical complex systems based on GNU/Linux (Allende, Mc Guire, Perez, Monsalve, & Obermaisser, 2021; Enabling Linux in Safety Applications (ELISA), 2020; Open Source Automation Development Lab contributors, 2019).

With the continuous development of GNU/Linux's functional characteristics and the continuous expansion of its application domains, the complexity of GNU/Linux is constantly increasing, and the Linux kernel is particularly prominent. As a result of this growing complexity, different system characteristics no longer behave as previously thought. Embedded systems are considered traditionally path-deterministic (Mc Guire et al., 2009), indicating that a function with the same input values does not always have the same execution path. Although simple systems (most simple applications running on single-core) are logically deterministic, resource-sharing and GPOS-based architectures are not. Okech et al. demonstrated in different publications that multi-core systems running the Linux kernel are no longer path-deterministic (Allende, Mc Guire, Perez, Monsalve, Fernández, et al., 2021; Mc Guire et al., 2009; Okech et al., 2015, 2014, 2013). They showed that the repeated execution of an application does not provide the same execution paths in the kernel space.

The diversity of function paths generates uncertainty in the execution of the application because the path that the application will follow in the execution is randomly decided. This feature is called path variability, which reflects the change degree in the system call execution path. Software must undergo rigorous testing and certification in safety-critical domains such as aviation, nuclear, and autonomous driving (Baron & Louis, 2021; Cummings & Britton, 2020; Jenn et al., 2020; Novak & Gerstinger, 2010). IEC 61508, a standard for the overall safety life cycle of electrical/electronic/programmable electronic systems(E/E/PE), required software testing techniques, such as "test coverage" (Bell, 1999; Fowler, 2022; Smith & Simpson, 2020). When performing test coverage-based verification of Linux, the uncertainty of the execution path will challenge the work because Linux has not been developed initially with safety applications in mind. Furthermore, it is impossible to force the execution of a certain path. Therefore, this becomes a critical factor in different industrial domains requiring exhaustive testing, such as the safety-critical domain (Allende, 2022; Allende et al., 2019; Enabling Linux in Safety Applications (ELISA), 2020;

Open Source Automation Development Lab contributors, 2019; Platschek et al., 2018; Procopio, 2020; Vágó, 2022).

Previous studies have shown that the execution path in the Linux kernel is non-deterministic, but this degree of non-deterministic changes under different kernel configurations and operating loads. This study delves deeper into the non-determinism of the Linux kernel to gain a deeper understanding of complex systems so that they can be appropriately tested in future work. It also explores the variation it undergoes depending on different scenarios to comprehend the significance of the inherent non-determinism of complex systems. To this end, this study takes the file system, the core component of the Linux kernel, as the focus of the research and analyzes in detail the relationship between system load, file system type, and the diversity of execution paths of system calls related to file operations. To effectively obtain the system call execution paths when the Linux system is running, this study designs and implements an online data acquisition system based on Ftrace and network file system (NFS) and integrates the component for cleaning path data in the system. At the same time, the MD5 algorithm generates a hash value for uniquely identifying the execution path of the Linux kernel function to facilitate statistical analysis.

In summary, our key contributions are outlined as follows:

- We design and implement a data acquisition system for the online collection of Linux kernel function execution paths, improving path data acquisition efficiency. Compared with offline data, the function execution paths obtained online can more effectively reflect the actual operating status of the system;
- We integrate an automatic data pre-processing component into the data acquisition system to clean the collected path data so that the system can process the path data in real-time, and then we use the MD5 algorithm to generate a 128-bit hash value for uniquely identifying the system call path;
- We design an experiment plan and explore the diversity of system call execution paths related to the file system from the two dimensions of system load and file system type. Analyzing the experimental data shows that both system load and file system type influence the path variability of the Linux kernel.

This study is structured as follows. Section 2 introduces the basic concepts to understand the research presented in the study. Section 3 presents the data collection system for system call paths in the Linux kernel. Section 4 presents the setup of the research. Section 5 explores the impact of system load on the path variability of system calls. Section 6 explores the impact of different file systems on the path variability of system calls. Section 7 presents the conclusion of the entire work.

2. Preliminaries

2.1. Basic concepts

This section introduces the basic concepts that help understand the performed study.

- We refer to path as the execution series of different kernel functions that starts with a specific system call invoked from a user-space application.

- Therefore, we define path non-determinism as the exhibition of different behaviour on the function executions of the same application with the same inputs and the possibility of following different execution paths with the same inputs.
- A system call follows different execution paths. Each different execution path is called a unique path. The number of unique paths represents how many execution paths a system call can follow.
- We refer to shared paths as those executed incidentally in different scenarios and are the same.
- There is always a most frequent path with the highest chance of appearing in execution, which we refer to as common path.
- There are also some paths with a low probability of appearing, which we refer to as rare path.

2.2. Ftrace

Ftrace, a widely used tracing tool in the Linux kernel, is usually used for tracing and debugging during kernel development. It helps kernel developers find what is happening inside the kernel, such as function call relationship, context switch, disable interrupt time, memory-mapped I/O, CPU power state transition, and kernel performance (Preisner, 2019; Rostedt, 2009a, 2009b, 2010). Kernel developers can accurately discover faults using Ftrace's various information on kernel operation. Ftrace is an abbreviation for function tracer, which can insert a segment of stub code at the start of all kernel functions (static instrumentation) through gcc performance analysis (profiling) and achieve the tracing of kernel functions by reloading the stub code. Consequently, during kernel compiling, *pg* option must be added to gcc, and the default stub code symbol name of gcc is mcount. During execution, Ftrace records the traced function information to a ring buffer in the kernel, and users can read the information recorded in the kernel buffer through debugfs (Beamonte et al., 2022). The structure of Ftrace is shown in Figure 1. Our study used Ftrace to trace the function execution paths of the system calls involving file system access in the Linux kernel online.

2.3. MD5 algorithm

The MD5 algorithm is a widely used hash function that accepts any length message as an input and transforms it into a fixed length (128 bits) output known as a hash value (Rivest, 1992). The MD5 algorithm was initially designed to encrypt hash functions, but it

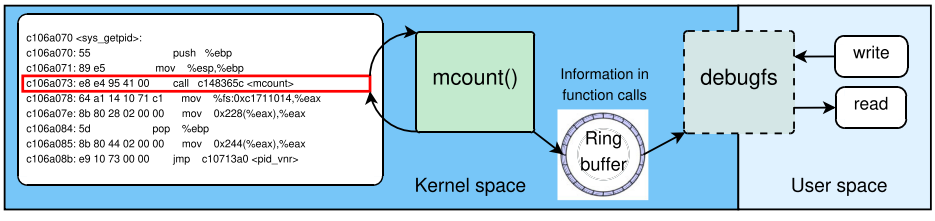


Figure 1. Ftrace architecture.

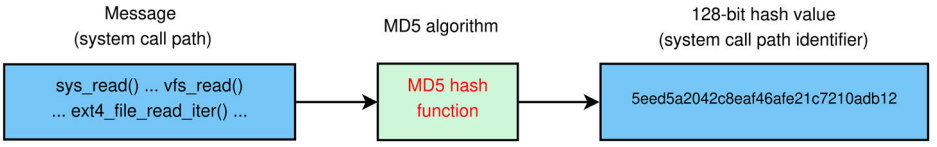


Figure 2. System call path encryption by MD5 hash algorithm.

was later proved flawed and decipherable; thus, it is no longer suitable for security authentication (Black et al., 2006; Lenstra et al., 2005). Nevertheless, the MD5 algorithm can still be used to check and verify data integrity (de Guzman et al., 2019; Sandeep & Abdulhayan, 2020), but it can only prevent accidental data corruption. As seen in Figure 2, in our study, the function execution paths of the system calls were used as the input of the MD5 algorithm, and the generated 128-bit byte sequences as the unique identifier of the execution paths of the system call (Allende, 2022; Allende, Mc Guire, Perez, Monsalve, & Obermaisser, 2021), as shown in Figure 2. This identifier can subsequently be used to analyse the diversity of system call function execution paths.

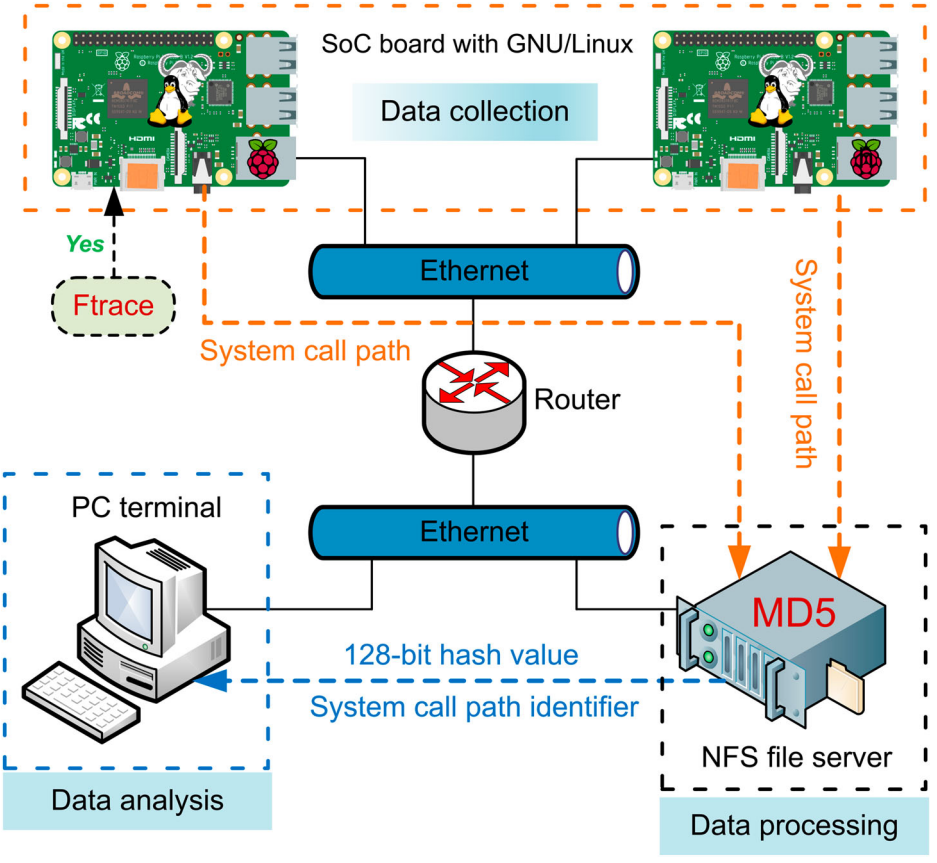


Figure 3. System call path collection system.

3. Data collection system design

To efficiently collect and process the function execution paths of the system call relevant to file systems in the Linux kernel, we proposed a design for path data collection, which includes three parts, namely, the data collection, data processing, and data analysis modules, as shown in Figure 3.

(1) The data collection module contains several System-on-Chip (SoC) based computing devices running GNU/Linux. To collect the information on the function execution paths of the system calls relevant to file systems in the Linux kernel, the Linux kernel image must be rebuilt and replaced first, and Ftrace must be enabled. In addition, Ftrace must be configured after restarting SoC-based devices. Finally, the network NFS client must be installed and configured to transfer the collected path information to the data processing module in real time.
(2) The data processing module runs the NFS server and is used to receive the path data from the data collection module in real-time. In addition, the data processing module must clean up the received path data and convert a complete system call execution path into a character sequence, which is then used as the input of the MD5 algorithm. Finally, the 128-bit hash values generated using the MD5 algorithm for uniquely identifying the function execution paths of system calls are saved to the database, where the database table at least includes the information on the file system type, name of system calls, hash value, and collection time.
(3) The data analysis module first obtains the collected data on the system call execution paths through multiple database access, then processes and analyzes the data using various tools and methods, and finally feeds back the results to the researchers.

4. Experiment setup

To analyse the path non-determinism of a Linux kernel-based system, we must record the kernel-space execution paths of a specific application under different scenarios. This section introduces the following: (i) the analysed system for this experiment, including the platform, executed application, and the Linux kernel; (ii) the method that is used to record the execution traces; and (iii) the scenarios that were examined in this study.

4.1. System description

The experiment was performed in a Raspberry Pi 3B, powered by an ARM Cortex-A53 quad-core processor running at 1.2 GHz and 1 GB RAM. Furthermore, the embedded system runs the Linux kernel version 4.19.75-cip11, a (CIP) Super Long-Term Support (SLTS) Linux version. The experiment was performed using a very simple application. We used a simple application for different reasons. We assumed that the observation of non-determinism in a simple application means that this characteristic will surely be maintained or even increased with more complex applications. However, it allows a deeper understanding of the system's behaviour and the non-determinism system it is executing. Finally, it facilitates a certain reproducibility of the experiment. The application opens one file to read its content and writes it to another. The code is shown as follows:

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/stat.h>
#include <fcntl.h>
#include <string.h>
#include <malloc.h>
#define File1 "/mnt/file_read"
#define File2 "/mnt/file_write"
int open_file(char *file_name, int flags)
{
  int fd;
  char retstr[100] = {0};
  fd = open(file_name, flags);
  if (fd == -1)
    exit(-1);
  return fd;
}
int main()
{
  int fd1, fd2, ret;
  fd1 = open_file(File1, O_RDONLY);
  fd2 = open_file(File2, O_WRONLY|O_TRUNC);
  int length = lseek(fd1,0,SEEK_END);
  lseek(fd1,0,SEEK_SET);
  char *result = (char*)malloc(sizeof(char)*length);
  char *res_str = (char*)malloc(sizeof(char)*length);
  ret = read(fd1, result, length);
  if (ret!= sizeof(unsigned char)*length)
    exit(-1);
  sprintf(res_str, "%s",result);
  ret = write(fd2, res_str, length);
  if (ret == -1)
    exit(-1);
  close(fd1);
  close(fd2);
  return ret;
}
```

4.2. Execution trace recording

Ftrace permits recording the execution trace in the kernel space. This highly configurable tool can be set to trace the kernel-space execution of a specific process. Ftrace allows us to investigate the path variability during the execution of this particular application. Therefore, we first mount the Ftrace to the file system in our experiment platform and configure the trace to "function_graph". We add the "noirq-info" to Ftrace's trace_option because we assume the interrupt (asynchronous event) should not impact the execution path.

We run the application repeatedly (no reboot) and record its execution trace in the kernel space. Once we have a defined number of paths, we calculate each MD5-hash to identify the different paths. Therefore, if two paths are equal, the hash result will be the same, allowing us to identify all the existing paths and determine their execution frequency.

4.3. Examined scenarios

The experiment starts with two scenarios:

- *Scenario 1:* Focuses on how the system load affects the path variability of system calls. The test programme and programmes, which are used to make the system load, are executed simultaneously. Therefore, we can get the execution paths of the system calls when the system suffers. The details are given in Section 5.
- *Scenario 2:* Examines the impact of different file systems path variability. The test programme and programmes that make the system load are executed in different file systems. Section 6 offers the detail.

5. Variability depending on system load

Previous studies (Mc Guire et al., 2009; Okech et al., 2015, 2014, 2013) demonstrated that a higher system load increases the probability of observing an increase in path variability. Consequently, these studies usually perform this analysis through a heavy CPU load. Linux is a fully preemptive operating system, indicating kernel tasks can also be preempted (Okech et al., 2013). However, in a system with different load levels, the preemption frequency may also be different. Therefore, preemption may cause context switching, producing a more significant number of unique paths when system calls are invoked (Okech et al., 2013).

This is because some resources need to attend to the task that is being traced. After all, they are being used for other tasks. The application's situation varies, as does the recorded function execution path. We focus on investigating how much the system load affects the path variability and whether there is a connection between the stress type and the application being executed. Associated with the concepts of *system load*is a concept of *load average*, which represents the current load status of the system. Our experiment mainly controls the system's load with some benchmark based on the value of *load average*. Note that the load average is different from the CPU utilisation. The former depends on the CPU queue length (Tenex load averages for July 1973 (No. 546), 1973), while the latter refers to the proportion of the time the CPU takes to run all the programmes, which reflects the current busy CPU. The running and waiting processes determine the CPU queue length. Thus, we controlled the number of threads generated by stressing benchmarks to control the system load. We assumed that the system load would affect the path variability of these system calls.

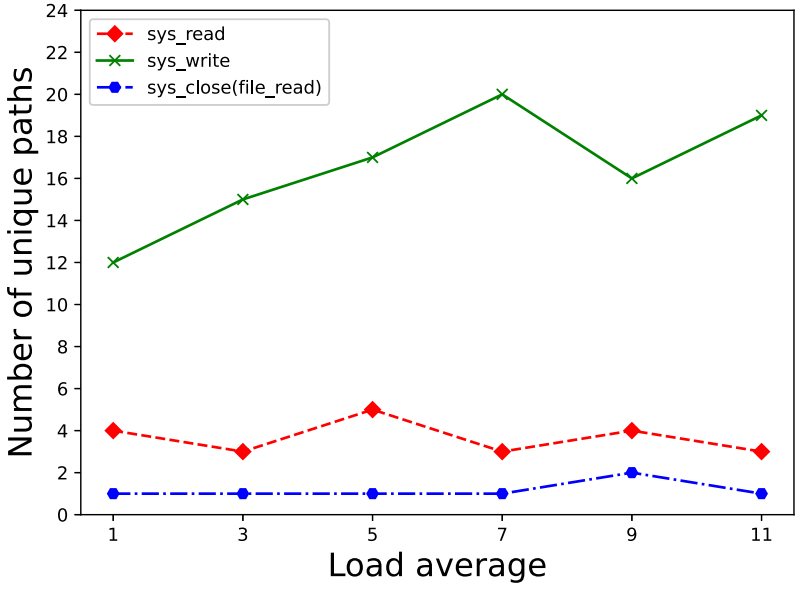
Therefore, this section studies the effect of system load on path execution. There are mainly two experiments shown as follows:

(1) Experiment focuses on the effect of different system loads made by the same benchmark.
(2) Experiment focuses on the effect of the same system load made by three different benchmarks that stress different resources (LMBench suite McVoy & Staelin, 1996).

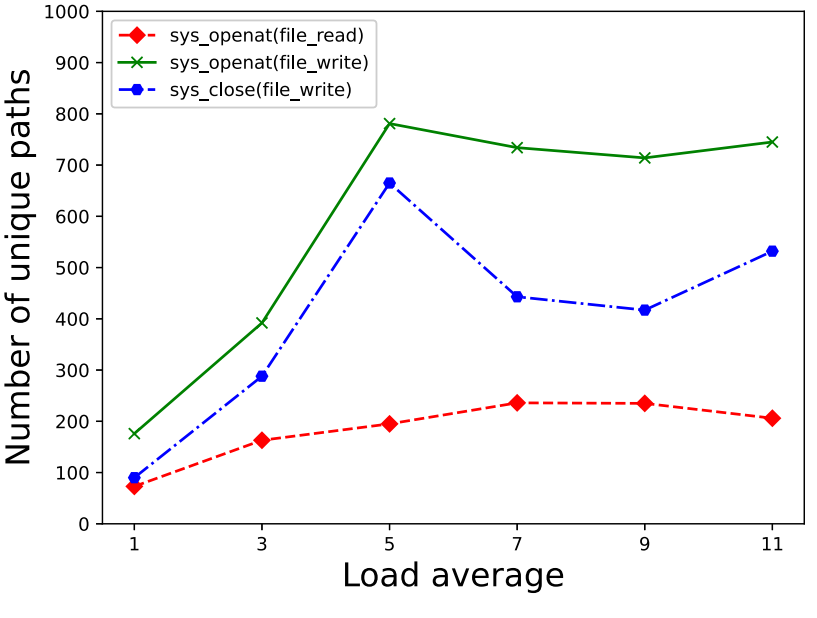
5.1. Effects of system load average

For the first experiment, to confirm whether the system load level affects the path variability level, we executed the test programme for six groups, each group formed by one thousand executions. The experiment is performed with different system load levels generated with *hackbench* stressing benchmarks. Hackbench is a benchmark tool that simulates process

communication by creating a group of client and server processes to send a specified number of bytes (Albanese et al., 2021; Zhang, 2008). Linux kernel developers commonly use it to stress the kernel scheduler. Because the benchmark is configured by setting the number of processes that stress the system, we can control the load average.



(a)



(b)

Figure 4. Distribution of unique paths.

Figure 4 shows the number of *unique paths* for each system call in each load average group. Figure 4 depicts two graphs to enhance readability, as the difference in unique paths varies based on the system call. *sys_read,sys_write,sys_close(file_read)* show low variance compared to other system calls. Figure 4(a) depicts that there is a maximum of 5 unique paths from 1000 executions, and the minimum is 3 for the *sys_read* system call. This shows the unique paths of system call *sys_read* are relatively stable no matter what the system load is. So do the system calls like *sys_close(file_read)* and *sys_write*. The number of their unique paths does not fluctuate significantly. In Figure 4(b), the number of unique paths of system calls *sys_openat(file_write)* and *sys_close(file_write)* is increased along with the value of the load average, which reaches the top when the load average is 5. When the value of the load average is greater than 5, the three system calls have different variation tendencies: the number of unique paths of *sys_openat(file_write)* varies between 700 and 800; *sys_close(file_write)* varies from 400 to 700; *sys_openat(file_read)* varies around 200. This study shows that system load affects the variability of system calls' execution paths. However, the effect of variability differs based on the system call and the executed application. Although variability in execution exists, this study shows that the same path is usually executed. Table 1 summarises the execution percentage of the most common paths.

We focused on the number of unique paths and the frequency of the common paths. The frequency of the common paths of system calls with different system loads is shown in Table 1. For system call *sys_read*, the frequency of its common paths is more than 99% with different system loads. When the CPU load average is 11, the system call *sys_openat(file_write)* has the lowest common path frequency, accounting for only 0.8% of the total. The percentage of common paths for system calls such as *sys_openat(file_read),sys_openat(file_write),sys_close(file_write)*, the percentage of common paths less than 50% indicating that they have more unique paths and thus have a higher path variability. Among these system calls, sys_openat(file_write) has more unique paths. We assumed this relates to the number of context switches during the system call. To confirm this, we isolated the paths interrupted by asynchronous events, such as task-switch, for each application. The result is shown in Figure 5.

Figure 5 shows that the number of paths with context switch that occurred between 1 and 4 times accounts for a considerable proportion of the total. There are a few paths where context switches occurred more than seven times. Therefore, we concluded that the number of context switches that occurred does not heavily affect the path variability. Still, the context switch influences the number of unique paths when system calls are invoked.

Table 1. Percentage of common paths.

	1	3	5	7	9	11
sys_read	99.6%	99.8%	99.6%	99.8%	99.6%	99.8%
sys_write	80.5%	75.6%	76.4%	79.0%	79.3%	76.7%
sys_openat(file_read)	47.7%	35.5%	16.1%	19.7%	18.1%	16.2%
sys_openat(file_write)	13.0%	7.6%	1.4%	1.7%	1.9%	0.8%
sys_close(file_read)	100%	100%	100%	100%	99%	100%
sys_close(file_write)	33.4%	12.7%	3.5%	8.3%	8.4%	6.8%

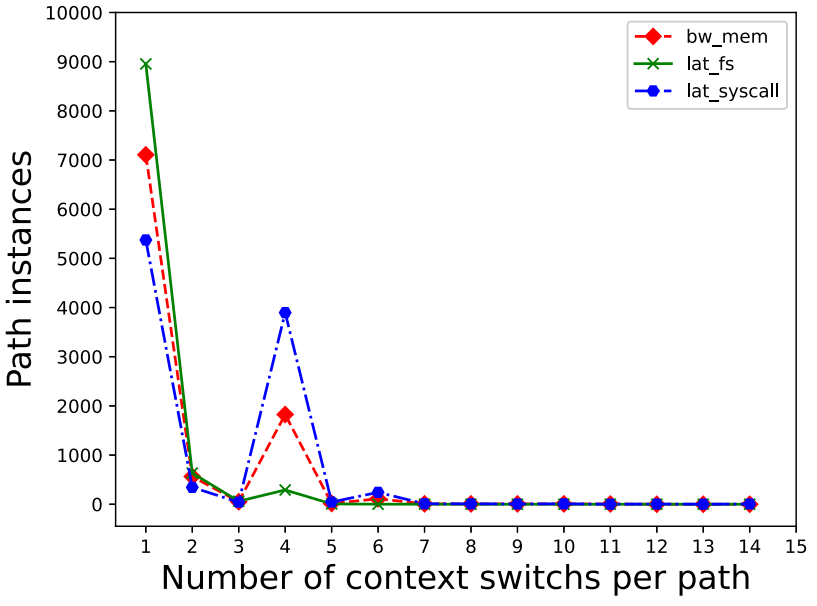


Figure 5. Path occurrence with context switches of sys_openat(file_write).

5.2. Effects of stressing different resources

The second experiment focuses on how the different benchmarks affect path variability when the system has an equal system load level. We choose benchmarks from LMBench. LMBench is a simple, portable, ANSI/C compliant micro-measure tool for UNIX/POSIX (Gravani et al., 2021; N. Li et al., 2019). It is developed to measure the performance of core kernel system calls and facilities, such as file access, context switching, and memory access. It is a suite of simple and portable benchmarks used to make bandwidth and latency benchmarks. It can satisfy our need for making the same system load average with different benchmarks.

In the experiment, we investigated whether there will be a difference when the system suffers the same stress caused by different applications. Thus, we use three different benchmarks to simulate these applications and ensure that the stress produced by these applications is equal by setting their parameters. The benchmarks we used are shown as follows.

- *bw_mem:* measures the bandwidth of the system. It is a memory-specific benchmark; thus, it can stress the system's memory.
- *lat_fs:* measures the file system create/delete performance. Its results include creating per second and deleting per second as a function of file size. It is used to stress the file system.
- *lat_syscall:* can time simple entry into the operating system via specified system calls. It can invoke expected system calls and time the execution process. We use it to create a load on the CPU.

These benchmarks can stress the different resources of the computer. Although their implementation mechanisms differ, they can be easily set up to produce the same system

load. The test and stress applications were executed 10,000 times, yielding 30,000 records. The data we obtained was divided into three groups, each group for each benchmark. Table 2 shows the count of the number of unique paths.

The second experiment shows the effect of different applications with the equivalent system load. Table 2 shows that the number of unique paths differs. There are 343 distinct paths with *bw_mem* for the *sys_read*. The number of distinct paths traced with *lat_fs* and *lat_syscall* is 7 and 20, respectively, demonstrating that applications with different load characteristics have a different influence on the number of distinct paths, as do the system calls such as *sys_write, sys_openat, sys_close*.

Table 3 shows the proportion of common paths in the total paths. According to the data we collect, *sys_close(file_read)* prefers to follow the same execution path when the system suffers the equal stress produced by different applications because its change rate is less than 1%. As for the other system calls, the frequency of their common path is significantly different when the system suffers equal stress. Therefore, we concluded that different applications running on the system influence the path distribution of the system call. In addition, the system calls such as *sys_openat(file_read), sys_openat(file_write), and sys_close(file_write)* have more unique paths because their frequency of common path is less than 20%. This shows that these system calls have a higher path variability, indicating that these system calls prefer to follow different execution paths when they are invoked.

6. Variability depending on the file system

A file system is a method of organising and retrieving files from a storage medium (Wirzenius et al., 2005). Linux kernel can support many file systems, and it uses a kernel function call Virtual File System (VFS), which provides a number of Application Programming Interface (API)s to operate it. We can modify the system configuration to use the desired file system type in Linux, but different file systems have various organisations and file management, which may differ in file operating-related system calls, and the differences could be expressed in path variability of system calls.

Table 2. Number of unique paths.

	bw_mem	lat_fs	lat_syscall
sys_read	343	7	20
sys_write	71	23	56
sys_openat(file_read)	2077	1364	1162
sys_openat(file_write)	3117	3336	3451
sys_close(file_read)	3	1	7
sys_close(file_write)	2249	6137	1781

Table 3. Frequency of common paths.

	bw_mem	lat_fs	lat_syscall
sys_read	87.32%	99.85%	99.66%
sys_write	64.55%	81.53%	74.41%
sys_openat(file_read)	9.27%	19.35%	17.72%
sys_openat(file_write)	3.72%	1.57%	2.94%
sys_close(file_read)	99.59%	99.95%	99.92%
sys_close(file_write)	10.01%	2.50%	7.47%

To examine the path variability on a Linux-based system depending on the configuration set, we investigated the impact of file system type on the execution path of an application. Therefore, to perform this non-determinism analysis, the application presented in Section 4 is executed in four file systems commonly found in Linux. The file systems that have been selected for this study are as follows: ramfs, ext2, ext3, and ext4. Ramfs is a file system based on ram under Linux whose structure is the simplest among the four file systems. Ext3 and ext4 are based on ext2 with more complex features and extends. Furthermore, ext4 is the default file system for the latest Linux kernel version. The application is based on reading and writing files so that we can record different paths because of the various file systems.

Based on Section 5 results, a higher CPU load provides a higher number of execution paths than an idle condition. Therefore, we generate high pressure for Raspberry Pi 3B in process scheduling by executing the hackbench command. Hackbench is a benchmark tool that simulates process communication by creating a group of client and server processes to send a specified number of bytes. By default, it will create ten groups, each with 20 clients and 20 server processes. Therefore 400 tasks can be created to schedule. With the argument "—l 100,000", each sender can pass 100,000 messages. In addition, pressuring Raspberry Pi 3B in Interrupt Request (IRQ) load by grep command can cause timer and disk access interrupts. After setting the load for the system, we executed the test programme in the Raspberry Pi and recorded the execution paths.

6.1. Examined features

With the recorded data, we can obtain different results that allow us to improve the understanding of the system's inherent variability. The results are presented by considering different points:

- *Shared path in each file system:* Checks the shared path that appeared in every four file system scenarios.
- *Unique path number in each file system:* The number of unique paths is the key to reflecting path variability. Examines the number of unique paths that have appeared in all executions.
- *Execution frequency of paths in each file system:* Based on the unique paths we analysed above, inspect the execution frequency of each unique path in the four file systems scenario.

6.1.1. Shared path in each file system

We analysed the shared path shown in all four scenarios to investigate whether the system call will share the same path in different file systems. Only the *sys_close* has the shared path appeared in all file system types in execution data. In addition, it has the highest frequency, which is close to 1K times in 1K executions, as shown in Table 4.

Table 4. Frequent of the shared path in each file system.

	ramfs	ext2	ext3	ext4
sys_close(file_read)	998	997	998	999
sys_close(file_write)	995	997	999	997

There is no shared path for *sys_read, sys_write, sys_openat(file_read), and sys_openat (file_write)* in any of the four file systems.

6.1.2. Unique path number in each file system

To investigate the path variability of different file systems, we collect the unique path numbers of each system call in each case. The specific result is shown in Table 5. System call *sys_read, sys_close(file_read), and sys_close(file_write)* have relatively fewer unique paths than *sys_write, sys_openat(file_read)*, and *sys_openat(file_write)*.

The frequency of occurrence is also an indicator of our analysis; the number of the unique call paths that only appeared once in test execution is shown in Table 6; compared with Table 6, those rarely appeared paths take the majority.

6.1.3. Execution frequency of paths in each file system

Although Table 5 shows the unique numbers of the path in different file systems, we can not see if some unique paths are shared between them. For more investigation, we analyse the frequency of each unique trace in each file system and draw the result in a heatmap way. As shown in Table 6, Figures 7, and 8, the X-axis represents the four file systems we select in our experiment, and the Y-axis represents the MD5 code of all the unique traces that the system call has in all executions. In addition, we use different colours to represent different frequencies in the figure.

Because the *sys_openat(file_read)*, *sys_openat(file_write)*, *sys_write* have a mountain of call paths that are difficult to illustrate graphically, we only present *sys_close* and *sys_write* for brevity, but it is enough to prove the influence of file system types on path variability. All four file systems have one common call path that has the highest frequency in execution, more than 96%.

In addition, most paths appeared less than ten times in all executions.

Table 5. Unique path numbers.

	ramfs	ext2	ext3	ext4
sys_read	7	5	6	5
sys_write	60	27	10	101
sys_openat(file_read)	41	49	55	45
sys_openat(file_write)	210	71	311	308
sys_close(file_read)	3	3	2	1
sys_close(file_write)	5	3	1	3

Table 6. Unique path numbers which only appear once in the execution.

	ramfs	ext2	ext3	ext4
sys_read	6	3	5	4
sys_write	45	18	9	75
sys_openat(file_read)	37	43	50	38
sys_openat(file_write)	168	54	294	248
sys_close(file_read)	2	2	1	0
sys_close(file_write)	4	1	0	2

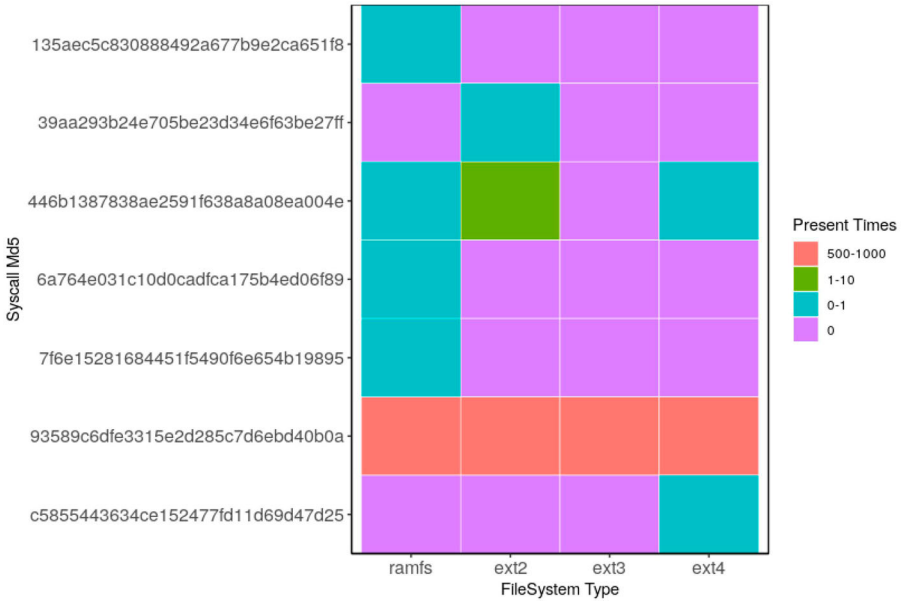


Figure 6. Heatmap of sys_close(file_write).

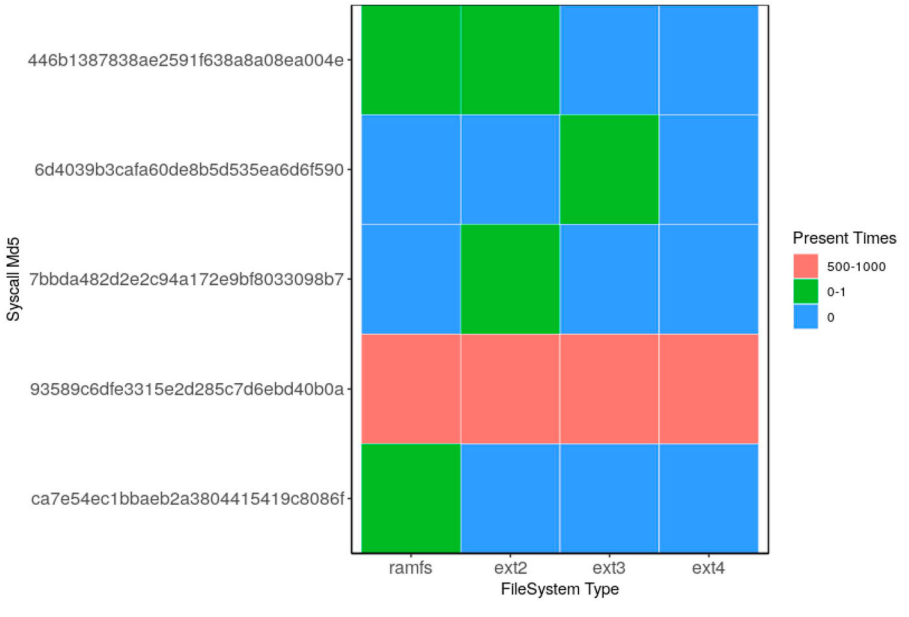


Figure 7. Heatmap of sys_close(file_read).

6.2. Results based on the file systems

The analysis of the shared path in each file system shows that not all file systems use the same call path. In the six types of system calls in the experiment, only *sys_close* has the

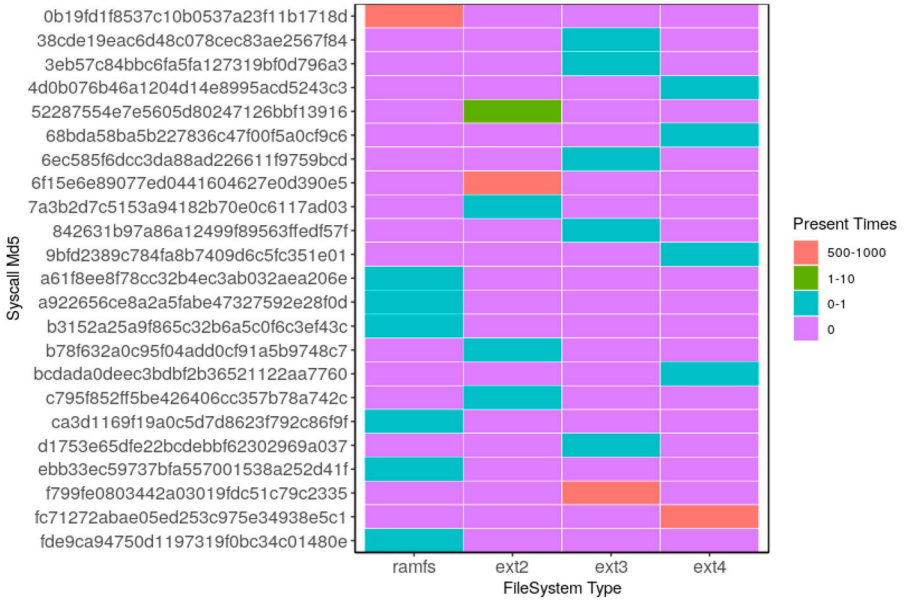


Figure 8. Heatmap of sys_read.

shared path used by all four file systems, indicating that different types of file systems may not share the same path in system calls.

On the number of path variability, Table 5 shows a significant difference between each file system in *sys_write* and *sys_openat(file_write)*. The *sys_close* is a unique system call that has fewer types of paths than the other system calls. The *sys_close(file_read)* in ext4 and *sys_close(file_write)* in ext3 only have one type of the path, implying that in 1K executions, they all follow the same path. Except for *sys_close*, compared Table 5 with Table 6, there are 66.7% minimum and 94.5% maximum unique call paths that appeared once in test executions, indicating that rare call paths account for the majority. In addition, Figures 6–8 show that most of the rare call paths that appear less than ten times in execution are scattered in each file system. Therefore, these type of file systems can significantly influence path variability and is caused by those rare call path, which has a very low probability of occurrence.

7. Conclusions

In this study, we investigated the impact of system load and different file systems on path variability. By conducting a series of experiments, we demonstrated that both system load and file system influence the path variability of the Linux kernel.

The variation tendency in the number of unique paths of the system calls tends to differ when the system has higher stress. Therefore the category of system calls and system status must be considered when calculating the test coverage of a system. Furthermore, the appeared variability is different depending on the applied stress. Different types of file systems also affect path variability. Consequently, variability differs according to the system's state, configuration, and the executed application. Our study could be helpful for the test

coverage-based work when doing certification for safety-critical systems as a reference. For developers who intend to use the Linux kernel for safety-critical systems, the selected kernel configurations and system running loads can affect the workload of system safety verification. The untested execution path during dynamic testing could have potential risks. Based on our results, the untested path numbers could be different when using different kernel configurations and run-time system loads. Therefore, the potential risk from the kernel's execution path level may also be affected by it.

In the future, the experimental methods still need to be improved further. If the online path collection feature can be integrated into the prototype verification system of a specific industrial product, the collected data will be more convincing. In addition, system calls that need to be observed cover all kernel components. The analysis results will be able to provide more convincing advice for the development (critical technology selection and system design) and certification of safety-critical systems. Furthermore, we will explore more factors that can influence path variability. Thus, we could obtain a model describing the inherent variability of complex software systems.

Acknowledgements

The authors would like to thank Nicholas Mc Guire and Imanol Allende for their thoughtful advice and review.

Disclosure statement

No potential conflict of interest was reported by the author(s).

Funding

This work was partially supported by National Key R&D Program of China under Grant No. 2020YFC0832500, Ministry of Education–China Mobile Research Foundation under Grant No. MCM20170206, the Fundamental Research Funds for the Central Universities under Grant No. lzujbky-2022-kb12, lzujbky-2021-sp43, National Natural Science Foundation of China under Grant No. 61402210, Google Research Awards and Google Faculty Award. This work was also partially supported by the Provincial Science and Technology Plan (Major Science and Technology Projects–Open Solicitation) (22ZD6GA048).

References

Albanese, G., Birke, R., Giannopoulou, G., Schönborn, S., & Sivanthi, T.2021). Evaluation of networking options for containerized deployment of real-time applications. In *2021 26th IEEE international conference on emerging technologies and factory automation* (ETFA) (pp. 1–8). IEEE Press.

Allende, I. (2022). Statistical path coverage for non-deterministic complex safety-related software testing.

Allende, I., Mc Guire, N., Perez, J., Monsalve, L. G.,Fernández, J., & Obermaisser, R. (2021). Estimation of Linux kernel execution path uncertainty for safety software test coverage. In *2021 design, automation & test in europe conference & exhibition (date)*. IEEE.

Allende, I., Mc Guire, N., Perez, J., Monsalve, L. G., & Obermaisser, R. (2021). Towards Linux based safety systems–a statistical approach for software execution path coverage. *Journal of Systems Architecture*, *116*, Article 102047. North-Holland, Inc.: Elsevier. https://doi.org/10.1016/j.sysarc.2021.102047

Allende, I., Mc Guire, N., Perez, J., Monsalve, L. G., Uriarte, N., & Obermaisser, R. (2019). Towards Linux for the development of mixed-criticality embedded systems based on multi-core devices. In *2019 15th european dependable computing conference (EDCC)* (pp. 47–54). IEEE.

Baron, C., & Louis, V. (2021). Towards a continuous certification of safety-critical avionics software. *Computers in Industry*, *125*, Article 103382. ISSN 0166-3615. https://doi.org/10.1016/j.compind.2020.103382

Beamonte, R., Ezzati-Jivan, N., & Dagenais, M. R. (2022). Execution trace-based model verification to analyze multicore and real-time systems. *Concurrency and Computation: Practice and Experience*, *34*(17), e6974. https://doi.org/10.1002/cpe.v34.17

Bell, R. (1999). IEC 61508: Functional safety of electrical/electronic/ programme electronic safety-related systems: Overview. In *IEE colloquium control of major accidents and hazards directive (comah)–implications for electrical and control engineers (ref. no. 1999/173)* (pp. 5/1–5/5). IET.

Black, J., Cochran, M., & Highland, T. (2006). A study of the MD5 attacks: Insights and improvements. In M. Robshaw (Ed.), *Fast software encryption* (pp. 262–277). Springer Berlin Heidelberg.

Borrego-Carazo, J., Castells-Rufas, D.,Biempica, E., & Carrabina, J. (2020). Resource-constrained machine learning for ADAS: A systematic review. *IEEE Access*, *8*, 40573–40598. IEEE Press. https://doi.org/10.1109/Access.6287639

Cummings, M., & Britton, D. (2020). Regulating safety-critical autonomous systems: Past, present, and future perspectives. In *Living with robots* (pp. 119–140). Elsevier.

de Guzman, L. B., Sison, A. M., & Medina, R. P. (2019). Implementation of enhanced MD5 algorithm using SSL to ensure data integrity. In *Proceedings of the 3rd international conference on machine learning and soft computing* (pp. 71–75). Association for Computing Machinery.

Diao, C., Zhang, D., Liang, W., Li, K. C., Hong, Y., & Gaudiot, J. L. (2023). A novel spatial-temporal multi-scale alignment graph neural network security model for vehicles prediction. *IEEE Transactions on Intelligent Transportation Systems*, *24*(1), 904–914. https://doi.org/10.1109/TITS.2022.3140229

Enabling Linux in Safety Applications (ELISA) (2020). [Web Page]. Available: https://elisa.tech/. Retrieved from https://elisa.tech/ ([Online]).

Fedullo, T., Morato, A., Tramarin, F., Cattini, S., & Rovati, L. (2022). Artificial intelligence-based measurement systems for automotive: A comprehensive review. In *2022 IEEE international workshop on metrology for automotive (metroautomotive)* (pp. 122–127). IEEE.

Fowler, D. (2022). IEC 61508 viewpoint on system safety in the transport sector: part 1–An overview of IEC 61508. *Safety-Critical Systems EJournal*, *1*(2.

Gravani, S., Hedayati, M., Criswell, J., & Scott, M. L. (2021). Fast intra-kernel isolation and security with iskios. In *Proceedings of the 24th international symposium on research in attacks, intrusions and defenses* (pp. 119–134). Association for Computing Machinery.

Hernandez, C., Flieh, J., Paredes, R., Lefebvre, C. A., Allende, I., Abella, J., Trillin, D., Matschnig, M., Fischer, B., Schwarz, K., & Kiszka, J. (2020). Selene: Self-monitored dependable platform for high-performance safety-critical systems. In *2020 23rd euromicro conference on digital system design (DSD)* (pp. 370–377). IEEE.

Jenn, E., Albore, A., Mamalet, F., Flandin, G., Gabreau, C., Delseny, H., Gauffriau, A., Bonnin, H., Alecu, L., Pirard, J., & Lefevre, B. (2020). Identifying challenges to the certification of machine learning for safety critical systems. In *European congress on embedded real time systems (ERTS 2020)*.

Lenstra, A., Wang, X., & de Weger, B. (2005). *Colliding x.509 certificates*. Cryptology ePrint Archive, Paper 2005/067. Retrieved from https://eprint.iacr.org/2005/067.

Li, N., Deng, R., Zhang, Y., & Zhou, H. (2019). Design discussion and performance research of the third-level cache in a multi-socket, Multi-core Microchip. In *Computer engineering and technology: 23rd ccf conference, nccet 2019, enshi, China, August 1–2, 2019, revised selected papers 23* (pp. 112–121). Singapore: Springer.

Li, Y., & Huang, Z (2022). Basics and Applications of AI in ADAS and Autonomous Vehicles. In Y. Li & H. Shi (Eds.), *Advanced driver assistance systems and autonomous vehicles: From fundamentals to applications* (pp. 17–48). Singapore: Springer Nature.

Manimuthu, A., Venkatesh, V. G., Shi, Y., Sreedharan, V. R., & Koh, S. C. L. (2022). Design and development of automobile assembly model using federated artificial intelligence with smart contract. *International Journal of Production Research*, *60*(1), 111–135. https://doi.org/10.1080/00207543.2021.1988750

Mc Guire, N., Okech, P., & Schiesser, G. (2009). Analysis of inherent randomness of the Linux kernel. In *Proc. 11th real-time Linux workshop*.

McVoy, L., & Staelin, C. (1996). Lmbench: Portable tools for performance analysis. In *Proceedings of the 1996 annual conference on usenix annual technical conference* (p. 23). USENIX Association.

Nidamanuri, J., Nibhanupudi, C., Assfalg, R., & Venkataraman, H. (2021). A progressive review: Emerging technologies for ADAS driven solutions. *IEEE Transactions on Intelligent Vehicles*, *7*(2), 326–341. https://doi.org/10.1109/TIV.2021.3122898

Novak, T., & Gerstinger, A. (2010). Safety- and security-critical services in building automation and control systems. *IEEE Transactions on Industrial Electronics*, *57*(11), 3614–3621. https://doi.org/10.1109/TIE.2009.2028364

Okech, P., Guire, N. M., & Okelo-Odongo, W. (2015). Inherent diversity in replicated architectures. arXiv preprint arXiv:1510.02086.

Okech, P., Mc Guire, N., & Fetzer, C. (2014). Utilizing inherent diversity in complex software systems. In *Proc. of the australian system safety conference (assc2014)*. Australian Computer Society, Inc.

Okech, P., Mc Guire, N., Fetzer, C., & Okelo-Odongo, W. (2013). Investigating execution path non-determinism in the Linux kernel. In *Proc. 14th real-time Linux workshop, lugano. osadl*.

Open Source Automation Development Lab contributors (2019, February). *SIL2LinuxMP: OSADL – open source automation development lab eG* [Web page]. Available: http://www.osadl.org/SIL2LinuxMP.sil2-linux-project.0.html. [Online].

Platschek, A., Mc Guire, N., & Bulwahn, L. (2018). Certifying Linux: Lessons learned in three years of SIL2LinuxMP.

Preisner, T. (2019). Efficient one-shot function tracing in the Linux kernel.

Procopio, G. (2020). Safety and security in GNU/Linux real time operating system domain. In *Proceedings of 6th international conference in software engineering for defence applications: Seda 2018 6* (pp. 245–254). Springer International Publishing.

Reghenzani, F., Massari, G., & Fornaciari, W. (2019, February). The real-time linux kernel: A survey on PREEMPT_RT. *ACM Computing Surveys*, *52*(1), 1–36. https://doi.org/10.1145/3297714

Rivest, R. L. (1992). The MD5 message-digest algorithm, internet request for comments. *Internet Request for Comments (RFC) 1321*.

Rostedt, S. (2009a, December). *Debugging the kernel using ftrace – part 1*. https://lwn.net/Articles/365835/.

Rostedt, S. (2009b). *Debugging the kernel using ftrace – part 2*. https://lwn.net/Articles/366796/.

Rostedt, S. (2010). Ftrace Linux kernel tracing. In *Linux conference Japan*.

Saddik, A., Latif, R., Ouardi, A. E., Elhoseny, M., & Khelifi, A. (2022). Computer development based embedded systems in precision agriculture: tools and application. *Acta Agriculturae Scandinavica, Section B–Soil & Plant Science*, *72*(1), 589–611. https://doi.org/10.1080/09064710.2021.2024874

Sandeep, K., & Abdulhayan, S. (2020). Implementation of data integrity using MD5 and MD2 algorithms in ioT devices. *PalArch's Journal of Archaeology of Egypt/Egyptology*, *17*(7), 7388–7395.

Smith, D. J., & Simpson, K. G. (2020). *The safety critical systems handbook: A straightforward guide to functional safety: Iec 61508 (2010 edition), iec 61511 (2015 edition) and related guidance*. Butterworth-Heinemann.

Tao, H., Chen, Y., & Wu, H. (2022). Theoretical and empirical validation of software trustworthiness measure based on the decomposition of attributes. *Connection Science*, *34*(1), 1181–1200. https://doi.org/10.1080/09540091.2022.2061424

Tenex load averages for July 1973 (No. 546) (1973, August). RFC 546. RFC Editor. Retrieved https://rfc-editor.org/rfc/rfc546.txt.

Vágó, R. (2022). Development of a safe architecture for embedded systems using Linux and Zephyr RTOS.

Wirzenius, L., Oja, J., Stafford, S., & Weeks, A. (2005). The Linux system administrator's guide [Web page]. Available: https://tldp.org/LDP/sag/html/filesystems.html.

Yong, B., Wei, W., Li, K. C., Shen, J., Zhou, Q., Wozniak, M., Polap, D., & Damaševičius, R. (2022). Ensemble machine learning approaches for webshell detection in internet of things environments. *Transactions on Emerging Telecommunications Technologies*, *33*(6), e4085. https://doi.org/10.1002/ett.v33.6

Zhang, Y. (2008). Hackbench. https://people.redhat.com/mingo/cfs-scheduler/tools/hackbenc.

Zhang, Y., Yu, J., Chen, Y., Yang, W., Zhang, W., & He, Y. (2022). Real-time strawberry detection using deep neural networks on embedded system (rtsd-net): An edge AI application. *Computers and Electronics in Agriculture*, *192*, Article 106586. ISSN 0168-1699. https://doi.org/10.1016/j.compag.2021.106586

Zhou, R., Zhi, P., Xu, X., Liu, Z., Li, X., & Zhou, Q. (2022). Artificial intelligence in engineering education in the case of self-driving vehicle curriculum. In *2022 IEEE 25th international conference on intelligent transportation systems (ITSC)* (pp. 341–348). IEEE Press.